# Logical Understanding or Repeating Patterns? The issue with measuring language model performance on simple arithmetic questions

**Nitay Hurvitz** **Liran Napadenski** **Daniel Baruch**

## Abstract

Language models show impressive performance on benchmarks involving multi-step reasoning and arithmetic, yet it remains unclear whether their success reflects genuine logical understanding or memorized token patterns. To probe this question, we introduce a dataset of single-digit word problems where the required arithmetic operator (addition or subtraction) is determined by a single verb in the prompt. This design isolates the logical-linguistic challenge from the numerical one. We evaluate OPT, GPT-2, Falcon, and the Pythia suite on those word problems and, for control, on equivalent explicit arithmetic questions. Results show that models trained on general text could not reach even decent performance on our implicit dataset, and scaling model size beyond 1B parameters does not improve results. In contrast models trained on STEM-heavy corpora (Falcon) achieve stronger performance. To capture the ability to resolve the right operator, we propose a new evaluation metric, the TS-index, which reveals gaps hidden by the overall accuracy. These findings suggest that pretrained models lack robust logical understanding of the language, and thus challenges the existing benchmarks for reasoning and math.

## 1 Introduction

Recent research has shown that language models can acquire significant arithmetic and logical abilities. With benchmarks such as GSM8K, SVAMP, and ASDiv widely used to probe this capacity, large language models have shown success of up to 97% accuracy on grade-school math problems (Zhong et al., 2025), and even small models trailing not too far behind (Liu et al., 2023). The picture is of course similar in research of reasoning abilities of language models, with existing benchmarks such as ARC (Clark et al., 2018) and DROP (Dua et al., 2019). These datasets typically emphasize multi-step reasoning, multi-digit arithmetic, and explicit mathematical formulations. The abilities of LLMs to solve math problems have been improved to such extent that it is now considered to be an accelerator of academic math (Frieder et al., 2024), (Ahn et al., 2024). However, there have been critics and thorough research questioning whether those models actually reached a generalized understanding of math, or is it just hidden overfit to the available math problems (Mirzadeh et al., 2025), (Zhang et al., 2024), (Ahn et al., 2024). In the work of Brown et al (Brown et al., 2020) for example, models of various sizes are tested on explicit arithmetic problems of 2 to 3 digits, and show very little success in models smaller than 100B parameters, even with few-shot context, emphasizing the concern of a hidden overfit.

For this reason and others, far less attention has been given to simple single-digit problems. In single-digit problems, the issue of distinguishing pattern recognition from actual logical reasoning becomes especially important, as the number of possible combinations is very small. When a model answers "5" to a "2+3" question, how can one tell if it had any understanding of the underlying arithmetic rather than connecting the tokens of 2 and 3 to the token of 5? But given a way to cope with this problem, single-digit question can shed light on the early stages of emergence: when and how models begin to handle simple numerical reasoning.

To tackle this, we introduce a dataset of single-digit, single-operation word problems where the arithmetic operator (plus or minus) depends on a single verb in the context of the question. The questions are phrased in such way that a mere change of a word can flip the operator in the question (See Table 1).

This dataset is an attempt to face the same issues as the authors of SVAMP (Patel et al., 2021). In their work, they present a dataset with several variations for each word problem (hence the V in SVAMP). The variation may or may not change the

| Prompt Template |
|---|
| *"In the company, there were {num1} workers before {num2} workers were {**verb**}. How many {items} are employed now?"* |

| Addition Verbs | Subtraction Verbs |
|---|---|
| *hired* | *fired* |
| *recruited* | *laid off* |
| *brought in* | *dismissed* |

Table 1: Example of a prompt from the implicit questions dataset and the possible verbs to be plugged in for it to be an addition question or subtraction question. For more variability the counted items (*workers*) could be replaced with other nouns such as *employees* or *consultants*

underlying arithmetic question, and thus challenge the understanding of the details of the questions. Our dataset follows similar rational, but it is designed to be more structured, in order to achieve a finer sensitivity to specific words. Furthermore, our aim is not only to test if the correct answer was given exactly, but wether the correct **operation** was inferred.

To help complete the picture of the emergence of logical abilities, we would like to test small models (1B-7B) on this problem set. The challenge in those problems will not be the arithmetic complexity of 2 and 3 digit problems, but rather linguistic and logical - resolving the mathematical operation out of natural language. By contrasting prompts that differ only by a single keyword, we aim to measure whether a model has internalized the link between language semantics and arithmetic operations, and how this ability emerges across model scales and throughout the training.

To have a reference point, the performance on this implicit questions dataset will be compared to performance on equivalent math questions in explicit form, that is to isolate the linguistic challenge from the arithmetic one.

## 2 Methods

### 2.1 Datasets

We construct a set of word problems templates covering addition and subtraction in varied narrative styles (e.g., food, animals, transport, employment). Each template includes a set of verbs, 3 verbs that implies addition and 3 that implies subtraction. Furthermore, each template contains variable slots for numbers and counted items (See Table 1). Prompts are phrased in natural, varied language as much as possible to reduce surface-level memorization. From each prompt template and verb choice we generate 20 different prompts, randomizing the choice of numbers (up to 10) and counted items (between 3 options). For the 10 templates in this Proof-of-Concept project, this results in 1200 prompts. Generating the same prompt with varying numeric values is in order to isolate the arithmetic difficulty from the linguistic challenge.

The reference dataset of explicit math problems is constructed in the same number range, over 3 templates with 2-4 variation each, resulting in 200 explicit questions prompts such as *"What is the sum of 5 and 3?"*. Performance on this dataset is meant to show the capability of the models to handle the numerical questions, so that the expected lower performance on the word problems could be attributed to the linguistic challenge alone.

### 2.2 Models

For general evaluation of the dataset, we run all prompts through OPT(125M and 1.3B)(Zhang et al., 2022), GPT2-medium (355M), Falcon3 (1B), and Pythia (1.4B, 2.8B, 6.9B) (Biderman et al., 2023). All of those models were evaluated on reasoning challenges such as ARC and scored $\sim 50\%$ or higher (evaluations for Falcon3 available in its HuggingFace page)

With the intention to explore the emergence of such logical abilities, the Pythia model suite was chosen. That is since it enables exploring the dynamics of the performance through the training, with a fraction of the resources that would have been required to train such models ourselves. The corpus of text used to train the Pythia models is a $\sim$1TB general-purpose dataset in English. It contains texts from diverse sources, that includes academic writing, which most certainly contains implicit and explicit numerical texts, but also contains prose and dialogues in which numerical texts are more sparse and probably implicit in form.

To maximize the performance and get meaningful results, we probe 15 evenly-spaced pretrained checkpoints across 3 of the largest available Pythia models, with number of parameters 1.4B, 2.8B and 6.9B.

### 2.3 Evaluation

We first evaluate the output by looking for the correct numerical answer among the first 20 generated tokens. But simply measuring accuracy rates across the entire dataset does not expose any new information on the capabilities of a model compared to existing benchmarks. In order to take full advantage of the structure of our new dataset, we would like to define a new metric that measures how well the model resolves both the addition verbs and the subtraction verbs in a given template.

In a trivial solution, one can set a threshold of accuracy, that defines success in a given template in the following way: for a threshold $X$, success in template $T$ will be defined as accuracy above $X$ in **both** addition and subtraction variants of $T$ (See Figure (1), the red dashed line being a possible threshold that only templates 1 and 7 barely cross). Averaging over many templates, one can get an accuracy-like statistic, expressing the portion of templates in which the accuracy of both operator variants was higher than the threshold. This can work well, but how should one decide on the value of the threshold $X$? What accuracy threshold should be considered success in a given prompt template? A possible and strict answer would be to set a high threshold, e.g $\sim 90\%$. This will for sure separate between poorly performing models and highly performing ones, but it will also wash out the interesting intermediate cases, as many models will not reach that standard even remotely. Setting a low threshold would cause a similar issue to the other side, as models may achieve a "high" template accuracy, when actually performing poorly, just passing the threshold. Furthermore, leaving the threshold to be the choice of the user makes the new dataset and metric inappropriate to be used as a benchmark, since the performance metric is inconsistent.

### 2.4 Template Success (TS-) Index

We present a new metric, *Template Success (TS-) Index*, in which there is no need to manually set a threshold. Instead, it will be set dynamically, in a way that creates a balance-point statistic, similar to the well known *h-index* metric that measures a scholar's productivity and citation impact. In a similar way to the balance in the *h-index* between number of publications and citation count, the *TS-index* will be the balance point between the threshold mentioned above and the "template accuracy"

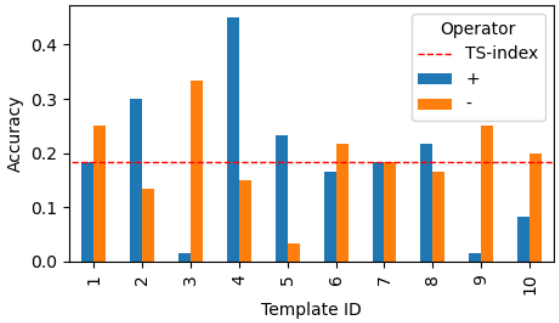


Figure 1: Accuracy rates per template and operator type on implicit questions. Templates 1 and 7 have an accuracy of at least 0.18 for both operators, and together they comprise a portion of 0.2 of the templates ("at least 0.18"). There are no 3 templates with shared accuracy greater than that, which makes 0.18 the TS-index. Results from Pythia 2.8B.

following from it. Since both the threshold and the accuracy are in the $[0, 1]$ range, the *TS-index* will also be in that range.

To define more rigorously, the *TS-index* on a dataset will be the maximum value $t \in [0, 1]$ such that at least $t$ of the templates has an accuracy of at least $t$ in **both** addition variants and subtraction variants.

For example, for our 10 templates, an index of $0.32$ means that more than 3 of the templates (i.e. 4 templates) has an accuracy rate of at least $32\%$ in the subtraction variants, as well as in the addition variants. Which is not a great success, but it does show some ability to distinguish subtraction from addition (See Figure 1 for visual explanation).

For comparison, a model that answers correctly on $64\%$ of the addition questions, and on $0\%$ of the subtraction questions, would have a an overall-accuracy of $0.32$, but a *TS-index* of exactly $0$. That is to show that unlike the overall accuracy, the *TS-index* is truly sensitive to the kind of performance we want to test - resolving the operators from the verbs and answering correctly to both directions.

A model that systematically fails to interpret the right operator from the verb should thus have a TS-index of no more than half of its performance on the explicit questions. The maximum of half is in the case where the model "chooses" the operator randomly and solves the underlying math question with the same accuracy as in an explicit question. For example a model that solves 100% of the explicit questions, and has no ability to resolve the right operator, thus it chooses randomly,

| Model | Size | Explicit Set | Implicit Set |
|---|---|---|---|
| OPT | 125M | 0.21 | **0.03** |
| GPT2 | 355M | 0.31 | **0.18** |
| Falcon3 | 1B | 0.99 | **0.58** |
| OPT | 1.3B | 0.19 | **0.09** |
| Pythia | 1.4B | 0.61 | **0.16** |
| Pythia | 2.8B | 0.63 | **0.188** |
| Pythia | 6.9B | 0.55 | **0.186** |

Table 2: Accuracy of the tested models on our 2 datasets, sorted by model size

could solve the implicit questions with at most 50% accuracy, thus its TS-index is also bound by 50%.

For more details and code implementation, see project's git repository.

## 3 Results

### 3.1 Overall Accuracy

Running both datasets on the various models shows that model scale is not necessarily the bottleneck in this type of simple math problems (see Table 2). Even though the models that are smaller than 1B perform poorly, it seems from examining the three Pythia models that scaling beyond 1B does not improve performance. Furthermore, Falcon3 shows strikingly high performance compared to all others, including a 6.9B-parameters model, which leads to the conclusion that size is indeed not the limiting factor. Since all models were trained on roughly the same scales of corpora, we should attribute this difference to the training data, while models like Pythia were trained on general text from various sources, Falcon3 was trained, among the rest, on a STEM corpus. In which the math part must have improved its capacity for such math questions.

### 3.2 Explicit Vs Implicit

Comparing the results between the implicit and explicit questions datasets, we learn that all of the models have a much better capacity for simple arithmetic problems than they are able to show in the implicit word problems. If phrased in explicit way, the models answer the questions with accuracy that may be 3 times larger. This shows in a decisive way that the implicit phrasing present a linguistic challenge for the models. Furthermore, going back to the sampled results in Figure (1), we see that some templates has an operator variant that is much easier for the models to solve, while the other variant have a vanishingly small accuracy, lowering the overall performance drastically. In figure 2 we can see that such dramatic differences between addition and subtraction variants are not seen in the results of explicit questions. This imbalance in implicit questions between the prompt variants will be compared between the various models in the next section.

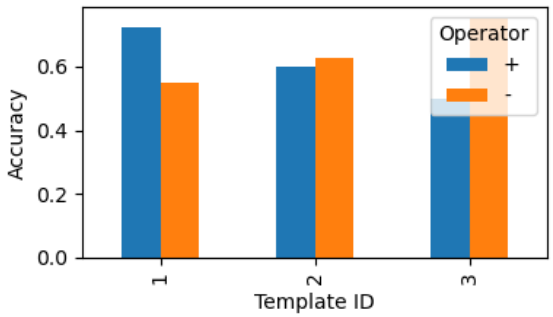


Figure 2: Accuracy rates per template and operator type in explicit questions. Results from Pythia 2.8B.

| Model | Size | Rel. Operator Imbalance |
|---|---|---|
| OPT | 125M | $1 \pm 0.00$ |
| GPT2 | 355M | $0.72 \pm 0.22$ |
| Falcon3 | 1B | $0.38 \pm 0.23$ |
| OPT | 1.3B | $0.70 \pm 0.38$ |
| Pythia | 1.4B | $0.31 \pm 0.24$ |
| Pythia | 2.8B | $0.53 \pm 0.33$ |
| Pythia | 6.9B | $0.51 \pm 0.24$ |

Table 3: Relative accuracy imbalance (mean and std-dev) between addition and subtraction questions from the same template out of the implicit questions dataset. 'Relative' means the absolute valued difference is divided by the higher accuracy value among the two operators.

### 3.3 Operator Imbalance

The imbalance of a model's accuracy on addition vs its accuracy on subtraction (see Table 3), is measured on each template, as the absolute valued difference between the addition accuracy and the subtraction accuracy. Then mean and standard deviation are taken across templates. This statistic washes out the average accuracy, so it is not a good metric on the dataset, but it comes to show that on each prompt, a model might have a tendency towards one of the operators, regardless of the verb (See Figure 1, in which templates 3 and 9 are shown to have very high imbalance, while template 7 is perfectly balanced). One cause for such imbalance could be the phrasing of the prompt - it might not

be neutral enough as we intended. For example, a question about the stock of a supermarket might be more easily interpreted as a question about stock depletion, hence subtraction. Though through a closer look at the results, template by template, we can see that different models might have opposite tendencies in **the same prompt** (See appendix A), hence the imbalance must have additional reasons. Looking at Table 3 we can see that any success that OPT-125M might had was incidental, since it had succeeded only in one of the operator variants of each prompt. Except for that, we can see great imbalance in the rest of the under-performing models, GPT2 and OPT-1.3B, implying that even if they succeed, they usually fail to flip the question based on the keyword. Interestingly, Falcon3, which performed at least 3 times better than any other model (on the implicit dataset), shows imbalance just as high as the Pythia models. In general, it seems that we did not manage to create a dataset nor find a model for which the accuracy is independent of the operator, or perhaps we got exactly the challenging task we aimed for.

### 3.4 TS-index

Surprisingly, the TS-index turned out to be very close to the accuracy (See Table 4). It is important to reiterate and emphasize here how independent the TS-index is from the accuracy, in order to appreciate how similar the results are. As stated before, an accuracy of up to 50% could result in TS-index of as low as 0%, though such a difference is indeed improbable. Of course, the TS-index is not completely independent from the accuracy, by its definition, but even though, a similarity between the accuracy and the TS-index does not render the new metric meaningless, as they measure different aspects of the performance.

And so, for the Falcon3 model, with the dramatically high accuracy, the TS-index did suppress its success. Showing that even though it answered correctly on 60% of the prompts, it had significantly less success in answering both variants of questions in each of the templates. And in that, avoiding exactly the pitfall we identified earlier of using the accuracy metric on this dataset. Compared to its 99% performance on explicit questions, the TS-index reveals that Falcon3 **does no cross the upper bound** of a model that chooses the operator randomly! (See end of section 2.4 for details on the upper bound).

In contrast, looking at GPT2, with TS-index of less than half of Falcon3, we can surprisingly say it **did** cross its own upper bound. Given its poor performance on math, it still outperformed a model with random choice of operators and equivalent math capabilities. Though for such low performance on our small template set, this conclusion should be taken cautiously.

| Model | Size | TS-index |
|---|---|---|
| OPT | 125M | **0** |
| GPT2 | 355M | **0.18** |
| Falcon3 | 1B | **0.43** |
| OPT | 1.3B | **0.1** |
| Pythia | 1.4B | **0.15** |
| Pythia | 2.8B | **0.18** |
| Pythia | 6.9B | **0.18** |

Table 4: TS-index per model, sorted by model size. Notice the similarity to Table 2 in all values except for Falcon3.

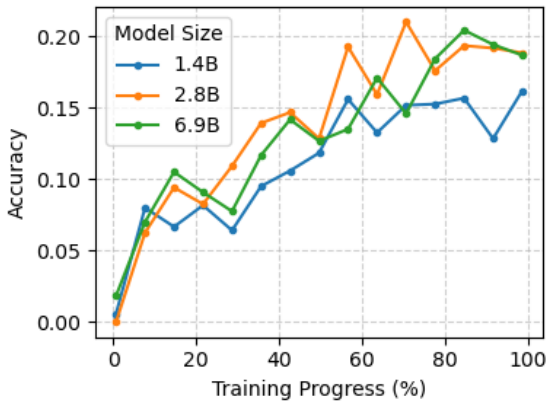


Figure 3: Accuracy of Pythia models on the implicit questions dataset throughout their training.

### 3.5 Training Dynamics

We saw in previous sections that the final performance of the Pythia models is quite similar in both accuracy and TS-index. We would now like to examine the dynamics of those metrics through the training. Repeating the runs of the 3 Pythia models on the implicit questions, on 15 snapshots taken throughout their training, gives the results presented in Figure (3) for overall accuracy, and Figure (4) for the TS-index.

Starting from the accuracy in Figure (3), it is immediately visible that the final snapshot is not unique - the learning curve is very similar between model sizes. Again, together with the high-performing Falcon3 model, this scale-invariance is

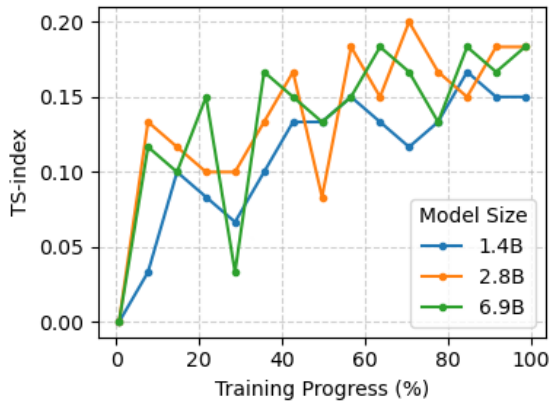


Figure 4: TS-index of Pythia models on the implicit questions dataset throughout their training.

most probably due to the absence the right signal in the training corpus. For the available signal, a model of 1B parameters is enough to learn what the training data has in it, and additional parameters does not lead to better generalization.

It is also showing large fluctuations in performance, a characteristic of small evaluation sets (even though there are 1200 distinct prompts, they are still generated from only 10 templates). Worth noting that the peak performance is reach already in approximately 60% of the training procedure.

Proceeding to the TS-index of Figure (4), it can be seen that larger fluctuations might be a caveat in the usefulness of the TS-index. But just as with other metrics, for a larger dataset we would expect the fluctuations to be suppressed.

## 4 Discussion

Our study set out to test whether language models genuinely acquire logical understanding of the verbs in word problems or merely exploit surface-level statistical patterns. To address this, we introduced two main contributions: (i) a dataset of single-digit word problems where the required operator is signaled only by a verb, isolating the linguistic–logical link from numeric computation, and (ii) the Template Success (TS-) index, a metric designed to capture operator-specific balance in model performance.

Our dataset isolates the linguistic trigger for arithmetic operations. The consistently low accuracies across multiple model families, including billion-parameter variants, demonstrate that operator resolution is a nontrivial challenge. These results suggest that strong performance on standard math benchmarks may not reflect genuine logical grounding, but instead memorization of patterns.

Regarding the evaluation metrics, the overall accuracy was found to obscure operator-specific weaknesses, which is the essence of the presented dataset. The TS-index provides a stricter lens, penalizing imbalances and thus capturing whether models truly understand the meaning of the different verbs in the sentence. For example, Falcon3 achieved high accuracy but a much lower TS-index, revealing it might have been "guessing" the underlying math question, without really understanding the role of the verbs in conveying the logical meaning. This validates the TS-index as a complementary evaluation tool beyond raw accuracy.

Analysis of Pythia checkpoints showed that increasing parameter counts did not improve performance, not in the final stage of training nor in the process of it, suggesting that scale alone is insufficient. On the other hand, Falcon3's advantage of STEM-heavy training data was also insufficient. It helped with solving math questions, as seen from its 99% accuracy on explicit questions, but the challenge to resolve the right operator remained significant, especially compared to its high score in explicit questions.

By disentangling linguistic cues from numerical difficulty, our dataset highlights a gap in current evaluation practices. The TS-index complements this by enforcing symmetry as a criterion for logical understanding. Together, these contributions provide a framework for probing whether models solve problems for the right reasons, offering a path toward more robust assessments of reasoning in future systems.

### 4.1 Limitations and Outlook

As a POC project, our study had more than a few limitations, that require taking our conclusions with a grain of salt. First, the dataset is based on a modest set of templates, which introduced variance in performance and very limited linguistic coverage. Second, we did not apply any fine-tuning, leaving open the question of whether operator resolution can be strengthened through targeted training. Furthermore, the models we tested were a very small and specific set. We cannot rule out that larger models, different architecture, or other training sets, could present much better understanding of language in the ways that were evaluated here.

Future work could address these limitations and further explore the emergence of logical abilities

in language models. Enlarging the dataset with additional templates, more diverse narrative domains, and broader lexical variation would suppress fluctuations in accuracy and yield more robust evaluation. Fine-tuning experiments could test whether models learn operator resolution more effectively when explicitly trained on this task, and whether scale influences this process—for example, whether larger models such as Pythia 6.9B align verbs with operations more quickly than smaller ones like Pythia 1.4B.

The missing link to an understanding of the logical meaning of verbs thus remains unknown. But hopefully, our attempt gave a fresh perspective on this challenge, and helped separate between the different capabilities needed for a true understanding of language in that context.

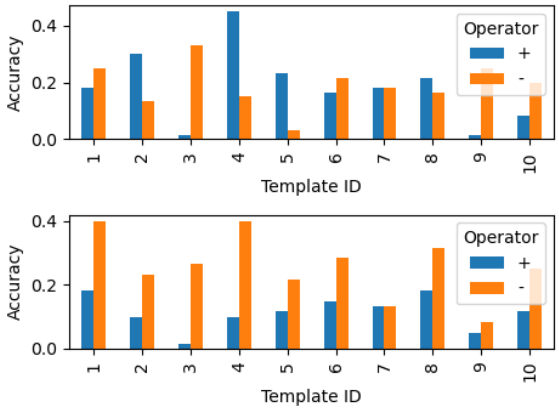


Figure 5: Imbalance in accuracy by operator for Pythia 2.8B (top) and Pythia 6.9B (bottom), broken down by template, to show different imbalances are possible for the same prompt template, see template ID's 4,5, and 8

## References


Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. 2024. Large language models for mathematical reasoning: Progresses and challenges. *Preprint*, arXiv:2402.00157.

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A suite for analyzing large language models across training and scaling. *Preprint*, arXiv:2304.01373.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. *Preprint*, arXiv:2005.14165.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *Preprint*, arXiv:1803.05457.

Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. 2019. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *Preprint*, arXiv:1903.00161.

Simon Frieder, Julius Berner, Philipp Petersen, and Thomas Lukasiewicz. 2024. Large language models for mathematicians. *Preprint*, arXiv:2312.04556.

Bingbin Liu, Sebastien Bubeck, Ronen Eldan, Janardhan Kulkarni, Yuanzhi Li, Anh Nguyen, Rachel Ward, and Yi Zhang. 2023.

Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. 2025. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. *Preprint*, arXiv:2410.05229.

Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are nlp models really able to solve simple math word problems? *Preprint*, arXiv:2103.07191.

Hugh Zhang, Jeff Da, Dean Lee, Vaughn Robinson, Catherine Wu, Will Song, Tiffany Zhao, Pranav Raja, Charlotte Zhuang, Dylan Slack, Qin Lyu, Sean Hendryx, Russell Kaplan, Michele Lunati, and Summer Yue. 2024. A careful examination of large language model performance on grade school arithmetic. *Preprint*, arXiv:2405.00332.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. Opt: Open pre-trained transformer language models. *Preprint*, arXiv:2205.01068.

Qihuang Zhong, Kang Wang, Ziyang Xu, Juhua Liu, Liang Ding, and Bo Du. 2025.


## A Imbalance Break Down

See figure (5), in which it is clear that imbalances are not a constant per template.